

Cloud-based Networking for Facebook-class Webapps/Backends

Computer Networks Project Report

Team Members: Ibad Ur Rehman, Aaqib Zahid & Sahil Latif **Course:** Computer Networks **Date:** Dec 2025

Table of Contents

1. Introduction
 2. Project Objectives
 3. Tools & Technologies Used
 4. VPC & Subnet Design
 5. Compute Layer (EC2, Load Balancing, Auto Scaling)
 6. Security Architecture
 7. DNS & CDN (CloudFront)
 8. Monitoring Setup (CloudWatch)
 9. Cost & Resource Summary
 10. Conclusion
-

1. Introduction

This project simulates a scaled, secure cloud backend infrastructure modeled after large-scale web applications such as Facebook. Rather than building the actual application logic, the focus is on the **networking and infrastructure layer**; the VPC design, compute scaling, security architecture, global content delivery, and monitoring systems that allow such applications to serve millions of users reliably.

A simple Node.js backend application was deployed to demonstrate the infrastructure, but the emphasis of this project is on how cloud networking principles (subnetting, routing, load balancing, auto scaling, firewalls, CDN, and monitoring) come together to form a resilient, secure, production-style system.

2. Project Objectives

- Design an isolated Virtual Private Cloud (VPC) with proper subnetting across multiple Availability Zones for high availability.
- Deploy a load-balanced, auto-scaling compute layer.
- Implement defense-in-depth network security using Security Groups and Network ACLs.

- Enable secure, low-latency global content delivery using a CDN.
- Implement monitoring, logging, and alerting for operational visibility.
- Demonstrate the entire system end-to-end with a working, publicly accessible application.

3. Tools & Technologies Used

Category	Tool/Service
Cloud Provider	AWS (Amazon Web Services)
Networking	Amazon VPC, Subnets, Route Tables, Internet Gateway, NAT Gateway
Compute	EC2 (t3.micro), Auto Scaling Groups, Launch Templates
Load Balancing	Application Load Balancer (ALB), Target Groups
Security	Security Groups, Network ACLs, VPC Flow Logs
CDN/DNS	Amazon CloudFront
Monitoring	Amazon CloudWatch (Dashboards, Alarms, Log Insights)
Application	Node.js (HTTP server)
Documentation	draw.io / diagrams.net, GitHub

4. VPC & Subnet Design

4.1 VPC

A Virtual Private Cloud named `FacebookVPC` was created with CIDR block `10.0.0.0/16`, providing 65,536 available IP addresses.

4.2 Subnets

Four subnets were created across **two Availability Zones** (us-east-1a and us-east-1b) to ensure high availability:

Subnet Name	CIDR Block	Availability Zone	Type
Public-Subnet-1	10.0.1.0/24	us-east-1a	Public
Public-Subnet-2	10.0.2.0/24	us-east-1b	Public
Private-Subnet-1	10.0.3.0/24	us-east-1a	Private
Private-Subnet-2	10.0.4.0/24	us-east-1b	Private

4.3 Internet Gateway (IGW)

An Internet Gateway (`FacebookVPC-IGW`) was attached to the VPC to allow resources in public subnets to communicate directly with the internet.

4.4 NAT Gateway

A NAT Gateway was deployed in `Public-Subnet-1` with an allocated Elastic IP. This allows instances in the private subnets to initiate outbound connections (e.g., for software updates) without being directly reachable from the internet.

4.5 Route Tables

Route Table	Subnets	Route
Public-RT	Public-Subnet-1, Public-Subnet-2	0.0.0.0/0 → Internet Gateway
Private-RT	Private-Subnet-1, Private-Subnet-2	0.0.0.0/0 → NAT Gateway

This design ensures that public-facing components (like the load balancer) are reachable from the internet, while backend compute resources remain isolated in private subnets.

5. Compute Layer

5.1 Application

A lightweight Node.js HTTP server was deployed on each backend instance. It serves a page displaying the instance's hostname and live system metrics (uptime, memory, CPU core count, load average), which also visually demonstrates load balancing when the page is refreshed.

5.2 EC2 Instances

- Instance type: `t3.micro` (chosen for AWS Free Tier eligibility)
- Deployed in the private subnets (`Private-Subnet-1` and `Private-Subnet-2`)
- No public IP addresses assigned ; only reachable via the internal network

5.3 Application Load Balancer (ALB)

An internet-facing ALB (`Facebook-ALB`) was deployed across both public subnets. It forwards HTTP traffic on port 80 to a target group (`Backend-TG`) listening on port 3000, with health checks configured to detect and route around unhealthy instances.

5.4 Auto Scaling Group

An Auto Scaling Group (`Backend-ASG`) manages the backend instance fleet using a Launch Template:

Setting	Value
Minimum capacity	1
Desired capacity	2

Setting	Value
Maximum capacity	4
Scaling policy	Target tracking on Average CPU Utilization at 50%

This allows the system to automatically add instances under load and scale back down during low traffic, balancing performance and cost.

6. Security Architecture

The project follows a **defense-in-depth** model, layering multiple independent security controls so that no single misconfiguration exposes the system.

6.1 Security Groups (stateful, instance-level)

Security Group	Inbound Rule	Purpose
ALB-SG	80, 443 from 0.0.0.0/0	Public entry point
Backend-SG	3000 from ALB-SG only; 22 from admin IP only	Restrict backend access to ALB traffic
Database-SG	3306 from Backend-SG only	Reserved for future database tier

6.2 Network ACLs (stateless, subnet-level)

- **Public-NACL** — allows inbound HTTP/HTTPS and ephemeral return ports; associated with both public subnets.
- **Private-NACL** — allows traffic from public subnets on the application port and SSH from the public subnet only (for a future bastion host); associated with both private subnets.

Using both Security Groups and NACLs together provides two independent firewall layers: one stateful (per-instance) and one stateless (per-subnet).

6.3 VPC Flow Logs

VPC Flow Logs were enabled on `FacebookVPC` and delivered to a CloudWatch Logs group (`/aws/vpc/flowlogs`). These logs record source/destination IP, ports, protocol, and whether traffic was accepted or rejected — used later for traffic analysis via CloudWatch Logs Insights.

7. DNS & CDN (CloudFront)

An Amazon CloudFront distribution was created with the ALB as its origin to provide global content delivery:

- **Origin protocol:** HTTP (internal AWS traffic between CloudFront and the ALB)
- **Viewer protocol policy:** Redirect HTTP to HTTPS (so end users always connect securely)

- **Cache policy:** CachingOptimized
- **Origin request policy:** AllViewer
- **WAF:** Not enabled, to keep the project within AWS Free Tier cost bounds; existing Security Group/NACL layers were considered sufficient protection for this deployment

This gives users a secure (HTTPS) endpoint, served from AWS's global edge network, with caching that reduces load on the origin ALB.

8. Monitoring Setup (CloudWatch)

8.1 Dashboard

A custom CloudWatch dashboard (Facebook-Backend-Dashboard) was created with widgets for:

- EC2 CPU Utilization (per instance)
- ALB Request Count
- Healthy/Unhealthy Host Count
- Network In/Out
- Auto Scaling Group desired capacity

8.2 Alarms

Three CloudWatch Alarms were configured, notifying an SNS topic (Backend-Alerts) by email:

Alarm	Metric	Condition
High-CPU-Alert	CPUUtilization	> 70%
Unhealthy-Targets-Alert	UnHealthyHostCount	> 0
High-Traffic-Alert	RequestCount	> 1000/min

8.3 Log Insights

CloudWatch Logs Insights queries were used against the VPC Flow Logs to identify top traffic sources, rejected connection attempts, and traffic distribution by destination port.

9. Cost & Resource Summary

Resource	Estimated Cost
EC2 (t3.micro x2, Free Tier eligible)	\$0 (Free Tier) / \$15/month after
NAT Gateway	~\$0.045/hour + data processing
Application Load Balancer	~\$0.0225/hour + LCU usage
CloudFront	Free Tier: 50GB/month, then ~\$0.085/GB

Resource	Estimated Cost
CloudWatch (Dashboard + Alarms)	Free Tier covers basic usage
Total (project duration)	~\$5–10

(Fill in actual billing dashboard figures before submission.)

10. Conclusion

This project demonstrates a complete, production-style cloud networking architecture: a segmented VPC across multiple Availability Zones, a load-balanced and auto-scaling compute tier, layered network security, global content delivery via CDN, and full observability through centralized monitoring and logging. While the application layer was kept intentionally simple, the underlying infrastructure mirrors patterns used by large-scale web platforms like Facebook, and could be extended with a database tier, CI/CD pipeline, and custom domain for a production deployment.